DEEP LEARNING

Adapting Foundation Models

Subhankar Roy

Department of Management, Information and Production Engineering (DIGIP)
University of Bergamo

Recap: What are Foundation Models?

- Foundation models (FMs) are very big/deep deep neural networks (eg, Transformers-based models) that have been trained on internet-scale datasets. Some examples:
  - CLIP was trained on 400M image-text pairs
  - BERT is trained on the BooksCorpus (800M words) and Wikipedia (2,500M words)
  - GPT-3 was trained on billions of tokens. See below:

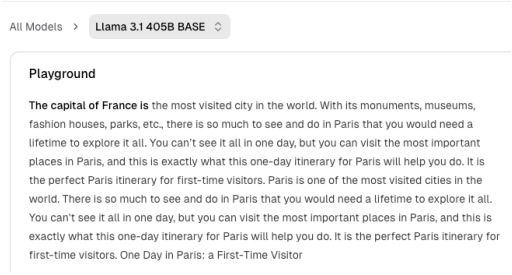
Dataset	Quantity (tokens)	Weight in training mix	Epochs elapsed when training for 300B tokens
Common Crawl (filtered)	410 billion	60%	0.44
WebText2	19 billion	22%	2.9
Books1	12 billion	8%	1.9
Books2	55 billion	8%	0.43
Wikipedia	3 billion	3%	3.4

Datasets used to train GPT-3

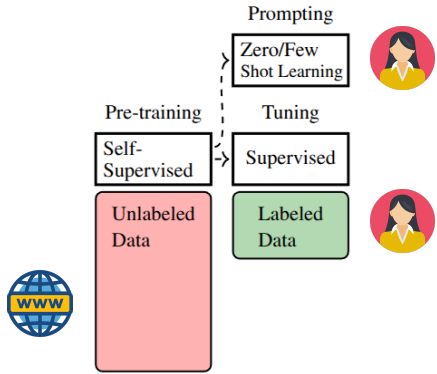
- Due to their massive pre-training, FMs are general-purpose learners that offer good generalization. Also pre-training is the most expensive phase of a model.

Why Adapt Foundation Models?

- While FMs are general-purpose learners, they have certain shortcomings when it comes to solving niche tasks.
- Shortcoming involve inadequate knowledge to handle certain tasks. For instance, Llama 3.1-base was trained by next token prediction. Although it is good at generating text, the output may not follow a desired user-friendly format. In this example the query was "The capital of France is", and the model wrote a lot about Paris, which may be unnecessary for an user.



Adapting Foundation Models



- In the pre-training phase we train a base model through a self-supervised objective (eg, next token prediction). This yields a strong model.
- Next, through prompting or tuning on curated data, we steer the behaviour of the base model towards our own needs (eg, a helpful assistant, no toxic output).

Today

- Pre-training vs adaptation vs prompting
- Different ways of adapting pre-trained models:
  - Full fine-tuning
  - In-context learning (ICL)
  - Parameter-efficient fine-tuning (PEFT)
- Applications in computer vision and natural language processing

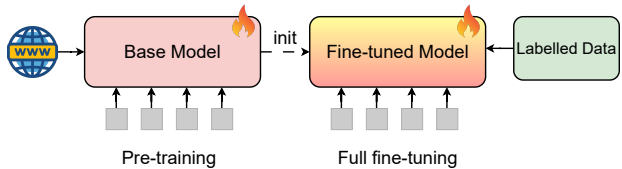
Table of Contents

1. Full Fine-Tuning
2. Adapting BERT Models
3. Adapting Generative Language Models
   - 3.1 Prompting
   - 3.2 Learning to Prompt
4. Parameter Efficient Fine-Tuning
   - 4.1 Prefix Tuning
   - 4.2 Adapters
   - 4.3 LoRA
5. Adapting Transformers for Image Classification
6. Adapting Diffusion Models
   - 6.1 Dreambooth
   - 6.2 Textual Inversion
   - 6.3 Prompt-to-Prompt

Full Fine-Tuning

Full Fine-Tuning

- A straightforward approach to adapt a pre-trained base model is via fine-tuning all the parameters of the model on the labelled dataset of a downstream task.



- Pros:
  - Offers maximum flexibility in learning.
  - High performance in the downstream task.
- Cons:
  - High memory footprint.
  - Prone to catastrophic forgetting, where all the previously learned information are erased.

Understanding Memory Requirements

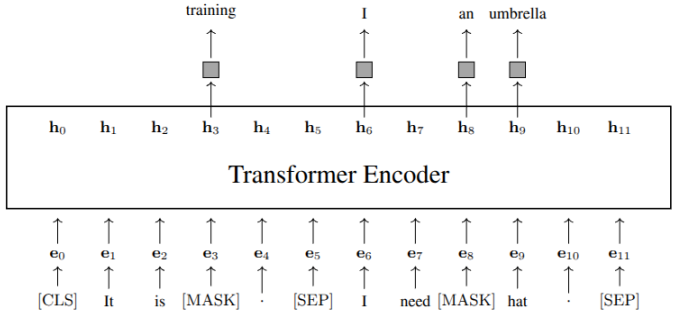
- How much memory (or VRAM) do we need to fine-tune a pre-trained model in full-precision (or FP32)?

Component	Stored Value	Bytes/parameter
Model weight	Parameter value	4
Gradient	Backprop result	4
Adam first moment (m)	Momentum	4
Adam second moment (v)	Variance	4
Total		16

- For a 7B parameter LLM, fully fine-tuning with Adam in FP32, the minimum memory requirement for parameter/optimizer $= 7 \times 10^9 \times 16 = 112$GB.
- Consumer GPUs have between 16GB-80GB of VRAM, not sufficient to fully fine-tune a LLM! Therefore, we need memory friendly fine-tuning methods.

Adapting  Models

Recap: Pre-training BERT



- BERT is an encoder-only Transformer model, that is pre-trained using a combination of two loss functions: Masked Language Modelling (MLM) and Next Sentence Prediction (NSP).
- These two are unsupervised (or self-supervised) pre-training objectives and does not require any manual labelling.

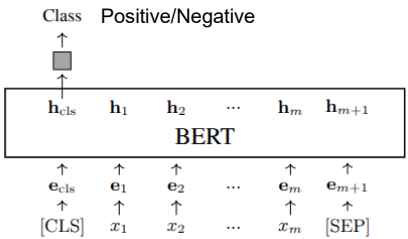
Adapting BERT Models

- Once a BERT model is pre-trained, it can then be used to solve NLP problems.
- But BERT models are not immediately ready for performing specific downstream tasks (eg, sentence classification).
- Therefore, we need a predictor model that will allow us to perform the downstream task while utilizing the pre-trained BERT model.

$$\mathbf{y} = \underbrace{\text{Predict}_{\omega}}_{\text{predictor}}\Big(\underbrace{\text{BERT}_{\hat{\theta}}}_{\text{pre-trained}}(\mathbf{x})\Big), \tag{1}$$

where $\mathbf{x}$ is the input sequence, and $\mathbf{y}$ is the output probability distribution; ω are the parameters of the predictor that we train on a downstream task; $\hat{\theta}$ are the parameters of the pre-trained BERT model that is typically kept frozen during adaptation.

Adapting BERT Models: Single-Text Classification



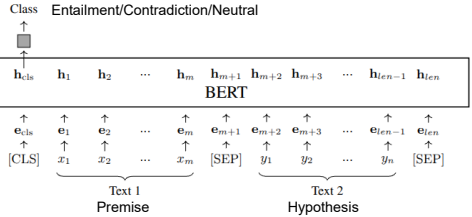
Training data

"The pizza was delicious. Friendly staff" - Positive

"The staff was rude, and made us wait for 1 hour" - Negative

- BERT is widely used for sentence classification. Given a sentence, we need to predict a distribution over the class labels (eg, positive/negative for food reviews).
- The representation $\mathbf{h}_{\text{cls}}$ corresponding to the [CLS] token is given as input to a prediction network (eg, a MLP+Softmax). During adaptation, the parameters of the prediction network is optimized using a labelled training dataset.

Applying BERT Models: Pair of Text Classification



Training data

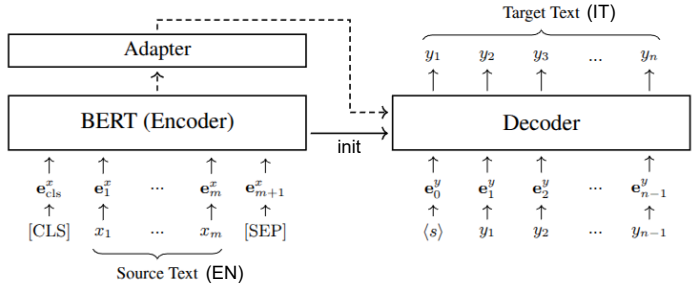
Premise: "A dog is running in the yard"
Hypothesis: "An animal is outside"
Label: Entailment

Premise: "A boy is reading a book"
Hypothesis: "A boy is watching the telly"
Label: Contradiction

Premise: "A teacher is talking to students"
Hypothesis: "The class is noisy"
Label: Neutral

- BERT can be used for tackling tasks such as Text Entailment Judgement that involve predicting a class for a pair of input sentences.
- Class labels: Entailment: hypothesis must be true if the premise is true. Contradiction: hypothesis must be false if the premise is true. Neutral: hypothesis may or may not be true.
- Similarly to before, we train a predictor on top of the [CLS] token.

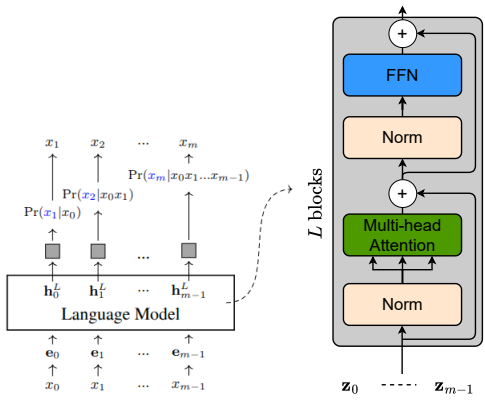
Adapting BERT Models: Encoding for Enc-Dec Models



- Pre-trained BERT model can be employed not only in classification tasks, but also in generative tasks, such as machine translation (EN $\rightarrow$ IT).
- BERT parameters are used for initializing both the encoder and decoder networks. Additional necessary layers in the decoder can be initialized from scratch.

Adapting Generative Language Models

Recap: Pre-training Generative LLMs



- A decoder-only Language Model is trained with next token prediction objective.
- Such a model stacks L transformer blocks.

Inference using LLMs

- During inference, we provide a context (or prefix text) to the LLM and ask the LLM to autocomplete. The LLM generates one token at a time, by appending the new token to the input. Below is an example.

Example

John seems happy today.
Question: Is this sentence grammatically correct?
Answer: _____

- To perform the task, the LLM is given the context "John seems happy today. \n Question: Is this sentence grammatically correct? \n Answer:"
- Based on the context, the LLM may output "Yes" if this text is the one with the maximum probability of prediction.

Prompting

Prompting

- Prompting refers to the method of providing an LLM with a specific input or cue to generate a desired output or perform a task, without the need to fine-tune any parameter. We can prompt an LLM to perform machine translation (EN $\rightarrow$ ZH) as:

Example

Translate the text from English to Chinese.
Text: The early bird catches the worm.
Answer: _____

- A well-crafted prompt can guide an LLM to generate more accurate, relevant, and contextually appropriate responses.
- The art of designing effective prompts to make better use of LLMs is called prompt engineering.

Prompting

- While there is no agreed upon prompt template, a popular way to format prompts is to write each input or output in a "name:content" style. In addition, specifying the task in a clear manner is beneficial. Below is an example.

Example

Task: Translation
Source language: English
Target language: Chinese
Style: Formal text
Template: Translate the following sentence: The early bird catches the worm.
Answer: _____

- Finding the right prompt for your downstream task is an iterative process.

In-context Learning

- Specifying a task in a clear manner is not always feasible or can be hard for the LLM to understand.
- In such cases, it is easier to provide a few demonstrations of the downstream task. Thus, "learning" occurs through the context, and hence called in-context learning.

Example

SYSTEM	You are a helpful assistant, and are great at grammar correction.
DEMO	You will be provided with a sentence in English. The task is to output the correct sentence. Input: There is many reasons to celebrate. Output: There are many reasons to celebrate.
USER	You will be provided with a sentence in English. The task is to output the correct sentence. Input: She don't like going to the park. Output: _____

Chain-of-Thought Prompting

- Certain problems can be complex for the LLM to execute, such as arithmetic problems.

Example without in-context learning

Q: Calculate the average of the numbers 2, 4 and 9.
A: The answer is 6. ✗

- A simple improvement is to add demonstrations of similar problems in the prompt.

Example with in-context learning

Q: Calculate the average of the numbers 1, 3, 5 and 7.
A: The answer is 4.
Q: Calculate the average of the numbers 2, 4 and 9.
A: The answer is 7. ✗

- It can be difficult for the LLM to answer correctly even with demonstrations.

Chain-of-Thought Prompting

- In chain-of-thought prompting (CoT)[1], not only the LLMs learn from the correspondence between questions and answers, but they gain more from detailed problem-solving steps that are used to derive the answers.

Example with CoT

Q: Calculate the mean square of the numbers 1, 3, 5, and 7.

A: Calculate the square of each number: $1^2 = 1, 3^2 = 9, 5^2 = 25,$ and $7^2 = 49$. Sum the squares, $1 + 9 + 25 + 49 = 84$. There are 4 numbers in total. Divide the sum by the number of items, $84/4 = 21$. The answer is 21.

Q: Calculate the average of the numbers 2, 4, and 9.

A: Calculate $2 + 4 + 9$, which equals 15. There are three numbers. Divide the total sum by the count, resulting in $15/3 = 5$. The answer is 5.

- The reasoning steps are highlighted in orange.

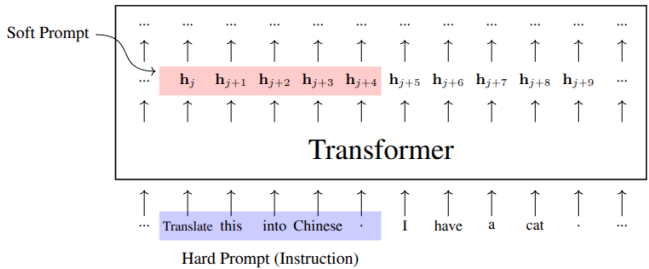
[1] CoT Paper

Learning to Prompt

Learning to Prompt

- Prompt engineering has the following drawbacks:
  - Designing high-quality prompts is inherently difficult and requires substantial manual effort.
  - Manual prompt design relies heavily on human expertise.
  - Prompts designed by humans can be complex and redundant, leading to longer inputs for LLMs and higher computational costs.
- As a solution, we want to learn the prompts such that:
  - We can automate the process of designing and optimizing prompts for LLMs.
  - Representing prompts beyond strings.
  - Make the prompts more concise and compact.
- This process of automatically learning to prompt is called as prompt optimization. We will learn only one specific kind, called soft prompting.

Soft Prompts

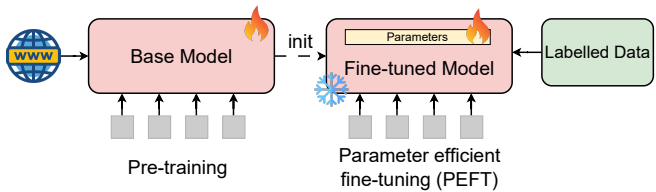


- All the prompts seen so far can be called "hard prompts" as they are provided to the LLM as discrete input tokens.
- The discrete input tokens get transformed into embeddings in the intermediate layers, which we call soft prompts.
- Soft prompts can be learned as standard parameters of the model through optimization. Unlike hard prompts, soft prompts are not human interpretable.

Parameter Efficient Fine-Tuning

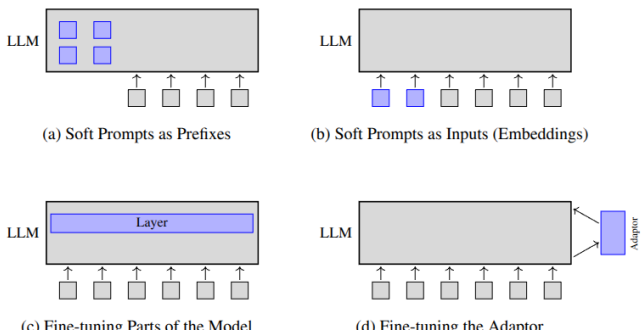
Parameter Efficient Fine-Tuning

- To overcome the limitations of full fine-tuning and prompt engineering, parameter efficient fine-tuning (PEFT) approach has been introduced that has the following characteristics:
  - Learns only a small fraction of the number of parameters of the base model, and keeps most of the pre-trained parameters frozen $\rightarrow$ memory friendly.
  - Eliminates the need to create hand-designed prompts.
  - Delivers competitive performance compared to full fine-tuning.
  - Prevents catastrophic forgetting.



Parameter Efficient Fine-Tuning

- PEFT approaches can be implemented in several different ways:
  - Approaches that optimize soft prompts (eg, prefix-tuning)
  - Approaches that optimize model parameters (eg, LoRA, Adapter)



(a) Soft Prompts as Prefixes

(b) Soft Prompts as Inputs (Embeddings)

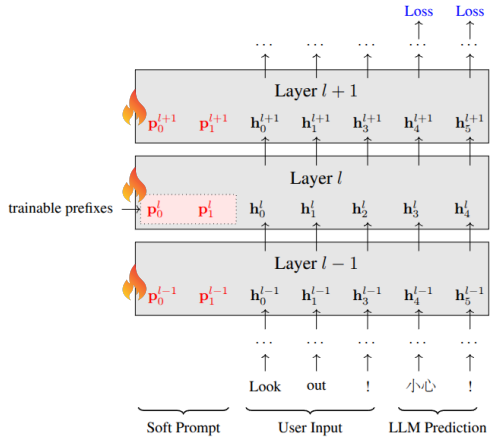
(c) Fine-tuning Parts of the Model

(d) Fine-tuning the Adaptor

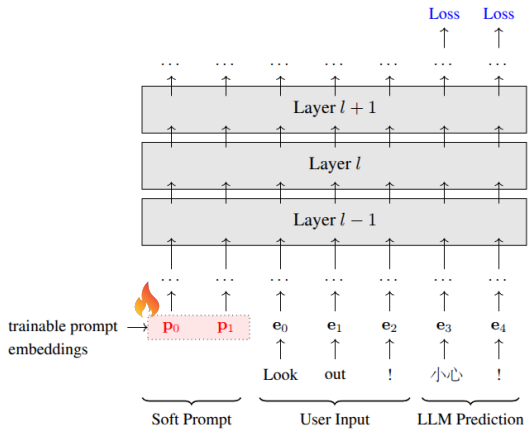
Prefix Tuning

- In prefix-tuning (or prefix fine-tuning) we append a series of trainable vectors (or prefixes or soft prompts) in the Transformer.
- During fine-tuning, we only need to learn the prefixes for encoding *task-specific knowledge* into the model. All the parameters + embedding layer of the model remains frozen.
- To fine-tune the prefixes, we use a standard cross entropy loss.
- There are two variants of prefix-tuning:
  - The prefixes can be inserted at the input of each Transformer layer.
  - The prefixes can be inserted as trainable embeddings at the beginning of the Transformer.
- These trainable vectors capture the essence of the downstream task.

Prefix Tuning

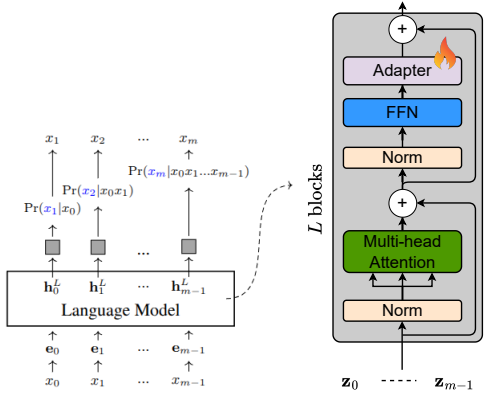


Variant 1



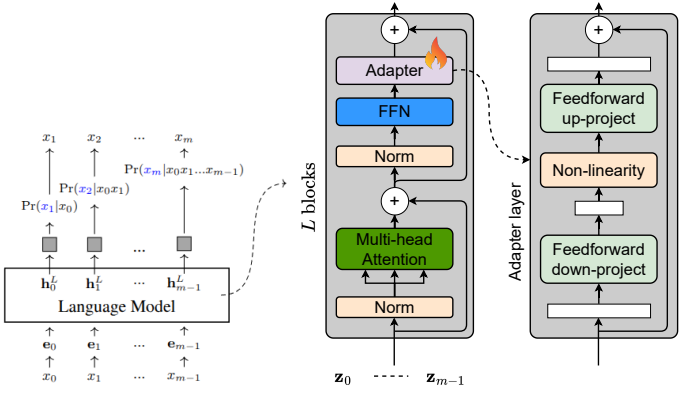
Variant 2

Adapters



- Adapter are new modules added between layers of a pre-trained networks.
- Only the parameters of the Adapter are fine-tuned during optimization.
- Can be used in both pre-trained BERT and decoder-only LLMs.

Adapters

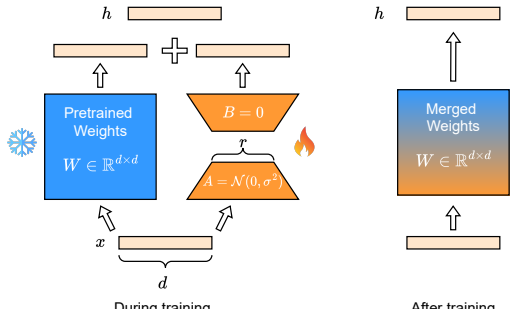


- In adapters[2], the features are down-projected with a linear layer to a lower dimension (or bottleneck), and then up-projected with a linear layer.
- Note the presence of a non-linear activation function.

[2] Adapter Paper

Low Rank Adaptation (LoRA)

- Low-Rank Adaptation (LoRA) is a technique that introduces lightweight low-rank matrices (A and B) to learn downstream task-specific information.



- During fine-tuning, only the low-rank matrices are fine-tuned and the rest of the model parameters are kept frozen.
- After fine-tuning, the low-rank matrices are merged with the original weights. This keeps the model size intact before and after fine-tuning, unlike adapters.

Low Rank Adaptation (LoRA)

- LoRA[3] leverages the following two key ideas:
  - Idea #1: We learn the updates (or new information) with a separate set of weights $\Delta W \in \mathbb{R}^{d \times k}$, and keep the pre-trained weights $W_0 \in \mathbb{R}^{d \times k}$ frozen:

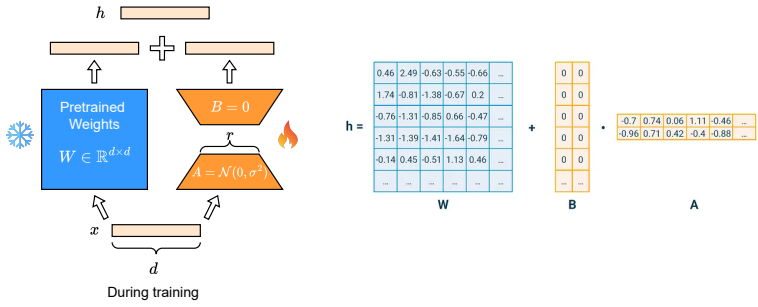
$$W = \underbrace{W_0}_{\text{freeze}} + \underbrace{\Delta W}_{\text{learn}}.$$

  - Idea #2: Instead of fine-tuning with a full-rank matrix $\Delta W \in \mathbb{R}^{d \times d}$, we fine-tune two low-rank matrices. That is, $\Delta W = BA$, where $B \in \mathbb{R}^{d \times r}, A \in \mathbb{R}^{r \times d}$, and the rank $r \ll d$. Because a high-rank matrix can be decomposed into two low-rank matrices with far fewer number of parameters (see next slide).
- The final output is:

$$h = W_0 x + \Delta W x = W_0 x + BAx = \underbrace{(W_0 + BA)}_{\text{merged}} x$$

[3] LoRA paper

Low Rank Adaptation (LoRA)



- Assume $d = 1024$ and rank $r = 4$:
  - W has $1024 \times 1024 \approx 1$ million params.
  - Instead of learning a full-rank $\Delta W \in \mathbb{R}^{1024 \times 1024}$, we learn two low-rank matrices $A \in \mathbb{R}^{4 \times 1024}$ and $B \in \mathbb{R}^{1024 \times 4}$. Verify the dimensions, $BA = \Delta W$.
  - A and B have $4 \times 1024 \times 2 \approx 8$k parameters, which is about 0.8% of the original parameters.

Understanding Memory Requirements of LoRA

- Assume we are learning LoRA that is equivalent to 0.1% of the total number of parameters. Base model = 7B parameters.

Component	Stored Value	Bytes/parameter	
		Base	LoRA
Model weight	Parameter value	4	4*0.001
Gradient	Backprop result	-	4*0.001
Adam first moment (m)	Momentum	-	4*0.001
Adam second moment (v)	Variance	-	4*0.001
Total		4	16*0.001

- Bytes/parameter = 4 (frozen weights) + 16 * 0.001 = 4.016 bytes
- Using LoRA in a 7B $\approx 7 \times 10^9 \times 4.016 = 28.11$ GB versus full fine-tuning that uses 112 GB.

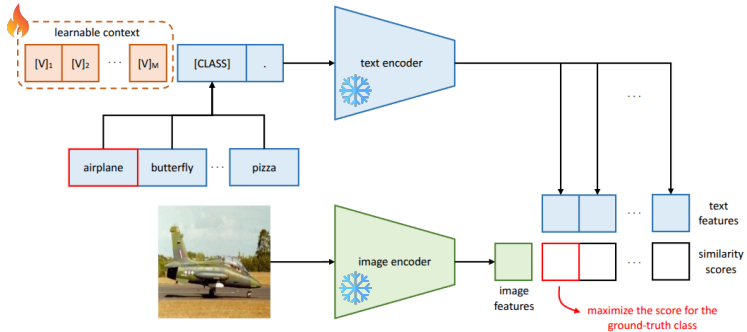
Adapting Transformers for Image Classification

Learning to Prompt in Vision Models

- The idea of learning to prompt using soft prompts can also be extended to Vision Transformers (ViT) and Vision-Language Models (VLMs).
- Two primary approaches:
  - Textual prompt tuning (TPT)[4], where we train soft prompts at the input to the CLIP text encoder.
  - Visual prompt tuning (VPT)[5], where we train soft prompts at the input or at intermediate layers of a ViT.

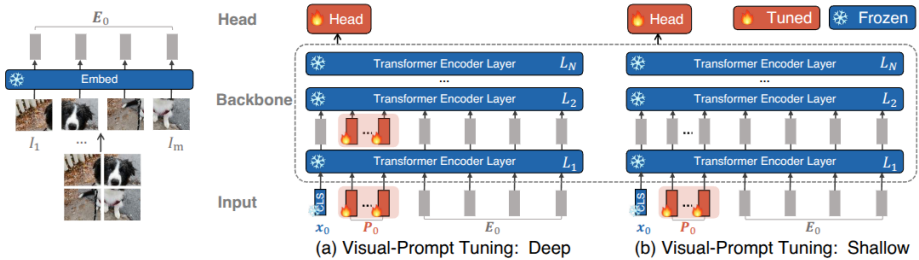
[4] TPT paper
[5] VPT paper

Textual Prompt Tuning for CLIP



- We introduce a set of M trainable vectors $V = \{V_1, V_2, \cdots, V_M\}$, where $V_i \in \mathbb{R}^d$ is a trainable vector.
- These vectors are appended to the CLASS names and given as input to the text encoder. During fine-tuning, we train only the trainable vectors, and keep the weights of both the image and text encoder frozen.

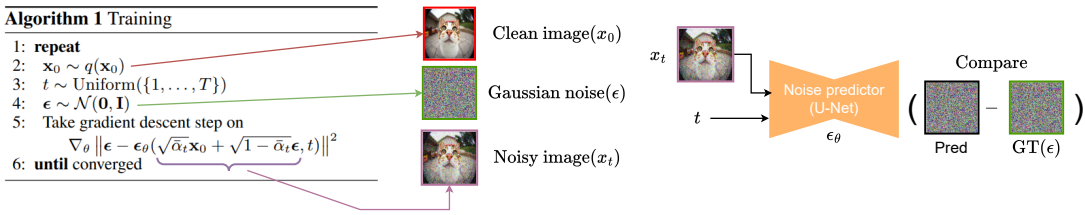
Visual Prompt Tuning for ViT



- Visual Prompt Tuning (VPT) has two variants:
  - Learnable prompts (or prefixes) are introduced as an input to each layer of the ViT, called "VPT-Deep".
  - Learnable prompts are introduced only at the input to the ViT, called "VPT-Shallow".
- In addition, it trains a classifier (or MLP) at the top of the representation of the [CLS] token. Rest are all frozen.

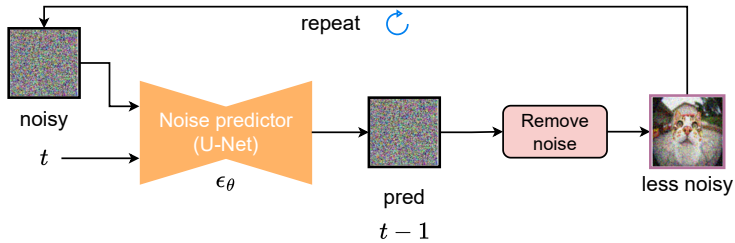
Adapting Diffusion Models

Recap: DDPM Training



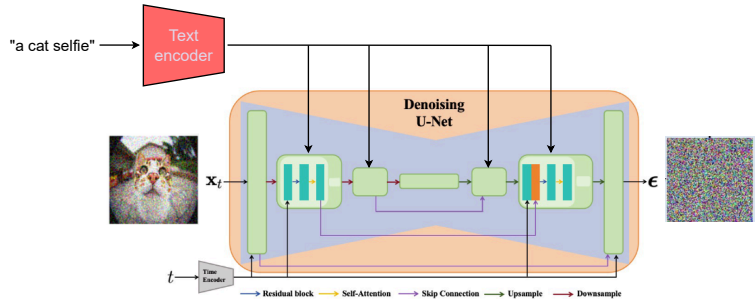
- In the forward diffusion process we add Gaussian noise to the clean image x_0, by sampling different values for timestep t. Higher the t, noisier is the image x_t.
- During reverse diffusion process, we ask the network (U-Net) ϵ_θ to predict how much noise has been added to the clean image. The parameters are trained by minimizing the predicted noise and the ground truth noise.
- Through training, the network ϵ_θ learns to remove noise from a noisy image.

Recap: DDPM Sampling/Inference



- During inference, we start from a Gaussian noise $x_T \sim \mathcal{N}(0, \mathbf{I})$.
- We ask the trained network to predict how much noise could have been added. We remove this noise and get a less noisy image.
- We give back the less noisy image back to the network and repeat this process for a pre-defined number of steps to get the final generated image.

Recap: Text-to-Image Diffusion Models



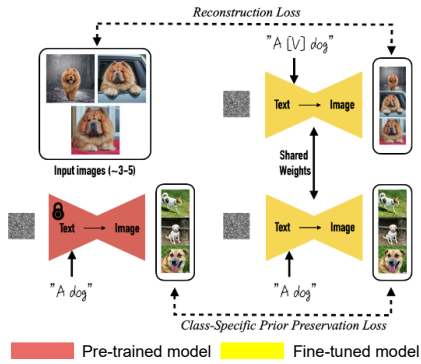
- The conditional diffusion models that incorporates text into the learning process are called text-to-image diffusion models (T2I diffusion models).
- Text (or caption) is encoded through a text encoder (eg, BERT) and injected into the U-Net through cross-attention layers.
- At inference time, given pure Gaussian noise and a starting caption, the model can generate images that adheres to the input caption.

Dreambooth

- While T2I diffusion models can generate different objects (eg, dogs) based on an input text, it can not consistently generate the same object or generate images of the object of interest (eg, your pet dog, not just any dog).
- Dreambooth is a technique that allows us to accomplish this by fine-tuning a pre-trained T2I diffusion models on a few images of the desired subject.
- Once the concept is learned, we can then create images of the subject in different contexts.

Dreambooth

- The approach is similar to TPT. We choose a rare token in the vocabulary, denoted by $[V]$, and create text prompts "A $[V]$ dog".
- The rare token used in this work is "sks", which appears rarely in the training data of the text encoder.
- Through fine-tuning the parameters of the U-Net + the embeddings of $[V]$, we bind the subject with the special token $[V]$. Note, the "sks" token is converted into its corresponding embedding from the look-up table.



Dreambooth

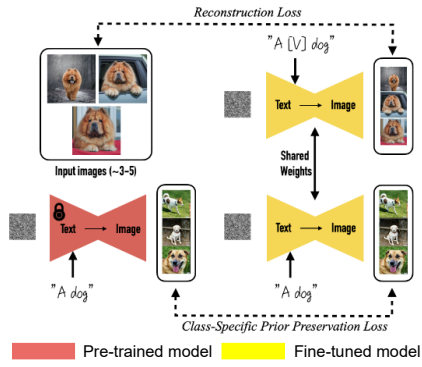
- Dreambooth[a] is trained with two losses:
  - Reconstruction loss:

$$\mathcal{L}_{\text{mse}} = \|\hat{x}_\theta(\alpha_t x + \sigma_t \epsilon, c) - x\|_2^2.$$

  - Prior preservation loss that encourages diversity in generation. It forces the fine-tuned model to generate images like a pre-trained model:

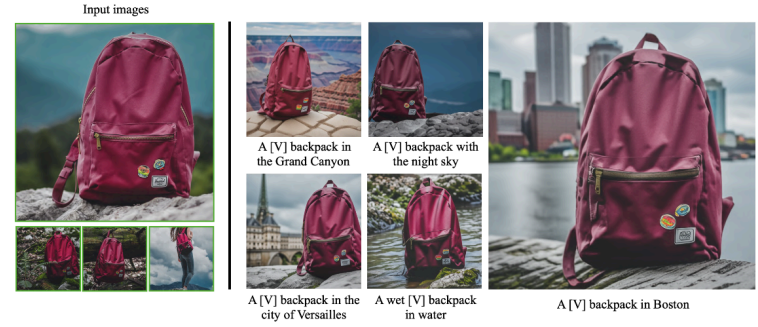
$$\mathcal{L}_{\text{prior}} = \|\hat{x}_\theta(\alpha_t x_{\text{pr}} + \sigma_t \epsilon, c_{\text{pr}}) - x_{\text{pr}}\|_2^2,$$

    where x_{pr} is an image generated by the pre-trained model with prompt c_{pr}.



[a] Dreambooth paper

Dreambooth

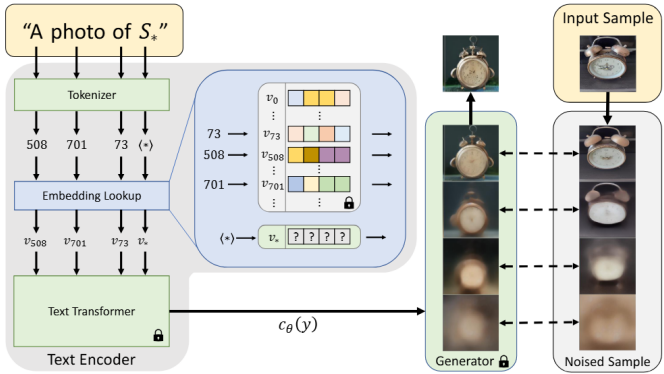

Input images
A [V] backpack in the Grand Canyon
A [V] backpack with the night sky
A [V] backpack in the city of Versailles
A wet [V] backpack in water
A [V] backpack in Boston
Input images
Top view
Bottom view
Side view
Back view
[V] cat seen from the top
[V] cat seen from the bottom
[V] cat seen from the side
[V] cat seen from the back

Textual Inversion

The goal of Textual Inversion[6] is similar to Dreambooth, in which we personalize a text-to-image diffusion model to generate novel images of the subject of interest.

[6] Textual inversion paper

Textual Inversion



- Similar to TPT for image classification and Dreambooth, we incorporate a special token S_* into the text prompt "A photo of S_*".
- We minimize the training loss of diffusion model to learn the embeddings v_*, and keep the parameters of the text encoder and the U-Net frozen.

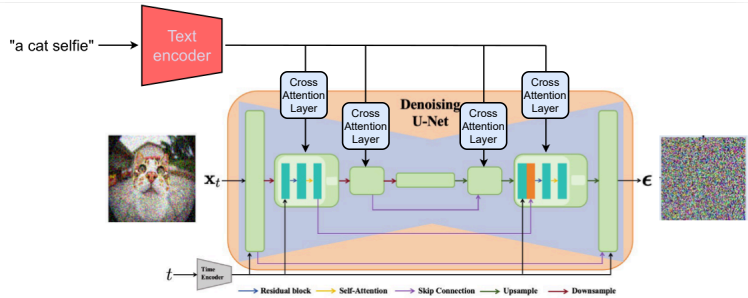
Textual Inversion

Prompt-to-Prompt

- In Prompt-to-Prompt[7], a user can edit small details of an image through text, without changing the scene layout. For instance, in the bottom left, we can change the style of the image from a real painting to children drawing.

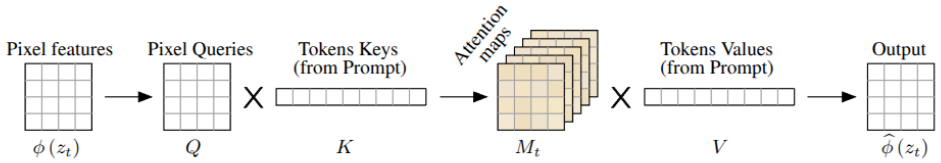
[7] Prompt-to-prompt paper

Recap: Cross-Attention Layers in T2I Diffusion Models



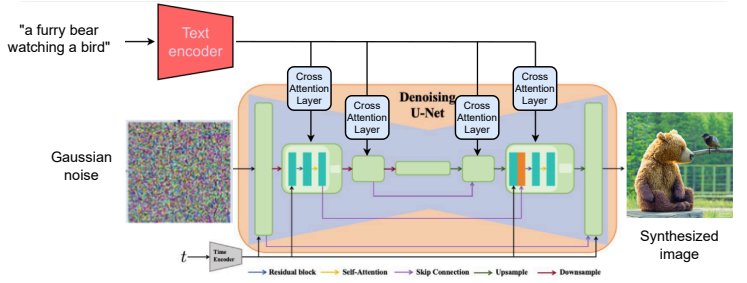
- Text information is injected into the U-Net through cross-attention (CA) layers.
- The K and V features comes from the text encoder, and the Q comes from the image (pixel) features.

Text-to-Image Cross Attention



- The three components in a cross-attention layer:
  - Q, derived from the pixel features.
  - K and V, derived from the text embeddings.
- Attention maps are computed using Q (from pixels) and K (from text) using the formula $M = \text{softmax}(\frac{QK^\intercal}{\sqrt{d}})$. A cell M_{ij} defines the weight of the value of the $j-$th token on the pixel i.
- Finally, the attention maps are multiplied with V (from text).

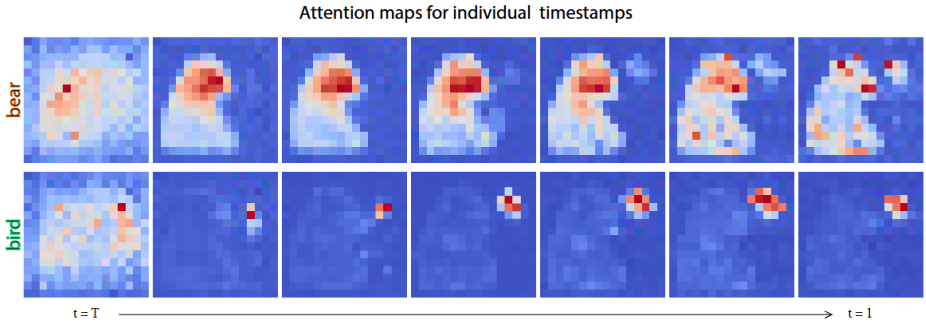
Text-to-Image Cross Attention



Cross-attention maps across text tokens

- Pixels are more attracted to the words that describe them. For example, the pixels of the bear are correlated with the word "bear". Similarly, for the object bird.

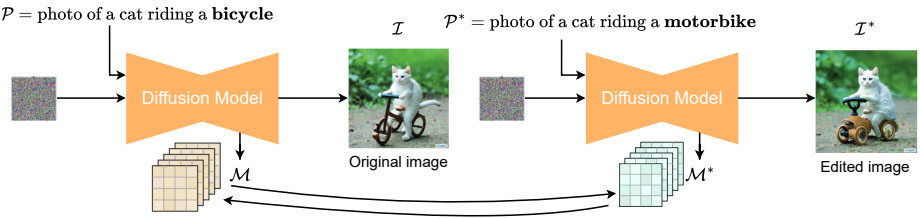
Text-to-Image Cross Attention



- An image is generated from text over T denoising steps.
- We notice that coarse scene layout is generated in the initial denoising steps, ie, $t \approx T$, which gets refined in the later denoising steps, ie, $t \approx 1$.

Prompt-to-Prompt: Cross Attention Control

- Let's focus on the editing operation: Word Swap.
- Let the original prompt and the modified prompt be $\mathcal{P}$ and $\mathcal{P}^*$, respectively.
- $\mathcal{P}$ and $\mathcal{P}^*$ differs by one word, which is bicycle $\leftrightarrow$ motorcycle.

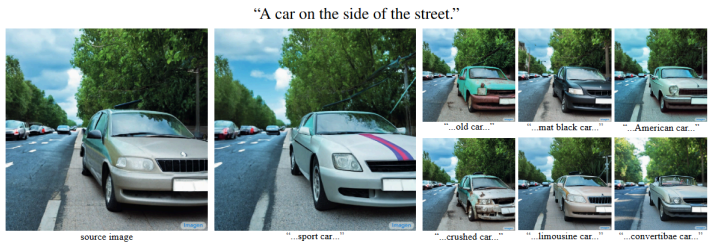


- In the initial denoising steps (ie, $t \approx T$), we use attention maps corresponding to $\mathcal{P}$, ie, $\mathcal{M}_t$. In the later denoising steps (ie, $t \approx 1$), we use the attention maps corresponding to the modified prompt $\mathcal{P}^*$, ie, $\mathcal{M}_t^*$.
- The intuition is that the coarse scene layout is determined by $\mathcal{P}$ and only the fine-grained details (eg, motorcycle instead of bicycle) is determined by $\mathcal{P}^*$.

Prompt-to-Prompt: Other Controls

Prompt refinement

Add new phrases to the modified prompt



Fader control

Control the extent to which an attribute influences the generated image

Considerations During Adapting Foundation Models

- During adapting the pre-trained base model, we need to consider the following.

1. **Cost vs. performance tradeoff**. Full fine-tuning = best performance. PEFT fine-tuning = near best performance, but cheaper.
2. **Data availability & quality**. For large datasets $\rightarrow$ full or instruction tuning. Small datasets $\rightarrow$ PEFT or prompt tuning.
3. **Compute & memory budget**. Full fine-tuning = hundreds of GBs of GPU memory. LoRA = way less GPU memory.
4. **Risk of catastrophic forgetting**. Tuning all the weights may erase general knowledge, or lead to catastrophic forgetting. PEFT reduces catastrophic forgetting.
5. **Extendability**. Full fine-tuning $\rightarrow$ models must be re-trained whenever the base changes. PEFT allows hot-swapping, and therefore rapid iteration.

References and Links

- For additional reading materials refer to the following links
  - "Foundations of Large Language Models"[8].
  - "Language Models are Few-Shot Learners"[9].
  - "A Survey on Prompt Tuning"[10].
  - "Prompt-based Adaptation in Large-scale Vision Models: A Survey"[11].
  - "Diffusion Model-Based Image Editing: A Survey"[12].

[8] https://arxiv.org/pdf/2501.09223
[9] https://arxiv.org/pdf/2005.14165
[10] https://arxiv.org/pdf/2507.06085
[11] https://openreview.net/pdf?id=UwtXDttgsE
[12] https://arxiv.org/pdf/2402.17525